

DraftPilot

AI-Powered Support Reply Co-Pilot — Comprehensive Technical Architecture & Milestone Report

FULL SYSTEM AUDIT, TECHNOLOGY STACK, IMPLEMENTATION RATIONALE & MILESTONES

Date: September 10, 2026 | Environment: Vercel Production + Supabase PostgreSQL | Monorepo: pnpm v10.34.5

EXECUTIVE ARCHITECTURE OVERVIEW

DraftPilot is an end-to-end, privacy-hardened support automation platform designed for customer support teams. It operates through a Manifest V3 Chrome Extension embedded seamlessly into Gmail, powered by a Next.js 14 edge cloud backend, resilient multi-tier LLM cascades via OpenRouter, and a granular SuperAdmin control center. 100% of automated test suites (626 tests) pass cleanly, and the platform is actively deployed to production.

1. WHAT WE BUILT

Comprehensive product architecture, components, and module breakdown

The DraftPilot platform is engineered as a unified monorepo containing four interconnected subsystems: the Chrome Extension Client, the Customer Web Application, the Edge API & AI Pipeline, and the SuperAdmin Root Control Center.

A. Chrome Browser Extension (Manifest V3)

- > **Native Gmail Thread Scanner:** Listens to Gmail URL changes and DOM mutators to extract incoming email threads, cleans customer bodies, strips quoted history, and identifies customer sender identities.
- > **Sidepanel Assistant UI:** Implements a non-blocking Chrome Sidepanel (chrome.sidePanel) containing one-click contextual draft generation, custom prompt guidance, knowledge base fact injection, and macro selection.
- > **One-Click Gmail Reply Insertion:** Directly targets Gmail compose windows and active draft boxes, dispatching synthetic InputEvents and text injection without DOM corruption.
- > **Client-Side Offline Fallback Synthesizer:** Contains an intelligent, rule-based support synthesizer capable of generating contextual, grounded drafts across refunds, billing, shipping, partnerships, and technical issues if network calls fail.
- > **Web-Handshake Bridge:** Injects a secured handshake content script onto draftpilot-web.vercel.app enabling real-time detection of extension installation status without user friction.

B. Customer Web Platform (Next.js 14 App Router)

- > **Interactive 4-Step Onboarding:** Guides new users through Gmail connection, extension download, creating their first macro, and testing demo mode, with automated celebratory milestone confetti.
- > **Interactive Demo Mode Sandbox:** A zero-risk testing sandbox allowing customer agents to simulate incoming tickets across 4 core support categories (Refunds, Order Status, Password Reset, Billing) with adjustable empathy tones.
- > **Analytics Overview Bento:** Real-time support telemetry displaying drafts generated, hours saved, macro adoption rate, average response speed, and date-range presets (7D, 30D, 90D, Month-to-Date).
- > **Macros & Knowledge Base Manager:** Full CRUD management for team macros with shortcut triggers (#refund), category tags, variable interpolation ({{name}}, {{customer_name}}), and keyword search.
- > **Team & Workspace Administration:** Role-based access control (Owner, Admin, Member), monthly team draft quotas (50 free vs 1,000 paid), and invitation workflows.
- > **Billing & Stripe MRR Manager:** Integrated tier comparison matrix, Annual vs. Monthly billing toggles, and feature gating.
- > **Custom PII Privacy Rules Engine:** Allows customers to define regex patterns and string replacements to mask proprietary customer identifiers before drafting.

C. SuperAdmin Control Center (/admin)

- > **Root Passkey Security Guard:** Constant-time comparison verification against dynamic database keys (platform_settings.root_passkey) and environment variables.
- > **OpenRouter Model Cascade Config & Playground:** SuperAdmin UI to select active models (z-ai/glm-5.3-flash, gemma-4-26b, llama-3.1-8b), inspect latency and reasoning tokens, test live prompts in a side-by-side sandbox, and monitor API spend.
- > **Global Macros & Multi-Tenant Knowledge Base:** Provisioning of cross-tenant policies and macros distributed across customer workspaces.
- > **Audit & PII Security Inspection Logs:** Real-time telemetry logging scrubbed PII occurrences, banned email interception, and rate-limit violations.
- > **System Feature Flags & Killswitches:** Granular toggles for BotID enforcement, OpenRouter cascade auto-fallback, maintenance mode, and beta modules.
- > **Account Switching & Console Locking:** Instant console locking and dedicated sign-out buttons allowing clean transitions between administrative inspection and user accounts.

2. WHAT WAS USED (TECH STACK & TOOLS)

Complete inventory of technologies, frameworks, APIs, and infrastructure

Layer / Component	Technology / Tool	Version / Spec	Primary Purpose in DraftPilot
Frontend Framework	Next.js (App Router)	v14.2.35	Server Components, dynamic routes, edge API hosting
UI Library & Styling	React 18 + Tailwind CSS	v18.2 / v3.3	Modern responsive dark-mode UI, layout bento boxes
Animation Engine	Framer Motion / Motion	v11.0 / v13.1	Smooth modal transitions, confetti, tab transitions
Chrome Extension	Manifest V3 + Vite	MV3 / Vite 5.4	Service worker, sidepanel, Gmail DOM content scripts
Language	TypeScript	v5.0.0	Strict end-to-end type safety across the entire monorepo
Package Manager	pnpm Workspaces	v10.34.5	Fast workspace orchestration, symlinked package builds
Database & Auth	Supabase (PostgreSQL)	v2.38.0 Client	Relational data, Row Level Security (RLS), JWT auth
LLM Gateway	OpenRouter API	API v1 REST	Dynamic multi-model cascade (GLM 5.3, Gemma, Llama)
Primary AI Model	z-ai/glm-5.3-flash	Reasoning / Flash	Fast, intelligent customer support reasoning & drafts
Bot Protection	Kasada BotID SDK	v1.4.5	AI bot/scrapper protection and human verification
Test Framework	Node.js Test Runner	node:test	Zero-dependency, blazing-fast native monorepo testing
Deployment & Host	Vercel Cloud Platform	Edge / Node runtime	Automated GitHub CI/CD, SSL, CDN distribution

3. HOW IT WAS BUILT & HOW IT WAS USED

Step-by-step pipeline execution, architectural flow, and interactions

Every user action moves through an orchestrated pipeline designed for zero data leakage, high fault tolerance, and sub-second rendering.

The 8-Stage Draft Generation Pipeline

- > **Stage 1 — Gmail DOM Ingestion:** When a support agent views an email in Gmail, gmail-detector.js scans the active thread, extracts sender names, cleans quoted histories, and dispatches the payload to sidepanel.js.
- > **Stage 2 — Client-Side PII Scrubbing:** Before any text leaves the browser, pii-scrubber.ts strips credit cards, SSNs, phone numbers, email addresses, and custom customer tokens, replacing them with generic tags like [REDACTED-EMAIL].
- > **Stage 3 — Edge Route Gateway Verification:** The extension calls POST /api/drafts/generate. The route verifies the caller Supabase JWT or Admin passkey, inspects the rate limiter (20 req/min), checks the banned email registry, and enforces monthly workspace quotas.
- > **Stage 4 — Knowledge Retrieval & Macro Matching:** The backend scans the workspace Supabase macros and document_chunks using keyword scoring, injecting the most relevant policies or macro text into the prompt context.

- > **Stage 5 — OpenRouter Model Cascade Execution:** The server attempts generation with z-ai/glm-5.3-flash under a 35-second timeout. If OpenRouter returns 429, 500, or times out, it automatically falls back to secondary models (gemma-4-26b-a4b-it or llama-3.1-8b-instruct).
- > **Stage 6 — Output Sanitization (cleanAiDraft):** The raw LLM output is parsed to strip <think> reasoning tags, markdown code blocks, header preambles ("Here is your draft:"), and normalize customer greetings (Hi {Name},) while preventing punctuation duplication.
- > **Stage 7 — Quota Recording & Heartbeat:** The backend atomically increments draft_count in the Supabase usage table and registers the extension heartbeat.
- > **Stage 8 — Sidepanel Delivery & One-Click Insert:** The cleaned draft is delivered to the sidepanel with an honest badge ([AI Generated] or [Fallback Template]). The agent clicks "Insert into Gmail Reply", which writes directly into Gmail compose container.

4. WHY WE USED IT (RATIONALE & TRADE-OFFS)

Design principles, engineering justifications, and defense-in-depth decisions

1. Why Chrome Sidepanel Instead of In-Page Gmail Iframe?

Traditional Gmail extensions inject large iframes directly into the Gmail DOM. This causes frequent CSS breakages when Google updates Gmail obfuscated class names, causes performance stuttering, and creates security issues. By adopting Chrome Manifest V3 Sidepanel (chrome.sidePanel), DraftPilot runs in an isolated browser process with guaranteed layout stability, full screen responsiveness, and zero interference with Gmail keyboard shortcuts.

2. Why OpenRouter Multi-Model Cascade Instead of Direct OpenAI API?

Relying on a single AI provider creates an existential single point of failure (outages, credit exhaustion, rate limits, or geographic latency). OpenRouter gives DraftPilot instant access to dozens of state-of-the-art models. Our automated cascade tries the high-intelligence reasoning model first (glm-5.3-flash), automatically failing over to ultra-fast instruct models (gemma-4-26b and llama-3.1-8b) if provider interruptions occur.

3. Why Dual-Layer PII Scrubbing (Client + Server)?

Customer privacy is non-negotiable. If PII scrubbing only lived on the server, raw sensitive data (passwords, credit cards, SSNs) would travel across the public internet to our servers. By scrubbing on the extension client first, sensitive tokens never leave the user local machine. The server then executes a second scrubbing pass as defense-in-depth before sending prompts to external LLM providers.

4. Why Supabase PostgreSQL with Row Level Security (RLS)?

Multi-tenant support tools must guarantee that Team A can never view Team B macros, drafts, or API keys. Rather than relying solely on application-level WHERE team_id = ... queries which can be forgotten in rapid development, Supabase RLS enforces multi-tenancy at the PostgreSQL database kernel level. Even if an anon client attempts a direct REST query, the database itself rejects unauthorized reads.

5. Why Native Node.js Test Runner (node:test)?

External testing frameworks like Jest or Vitest carry significant dependency overhead, complex Babel/esbuild configurations, and slow execution times in monorepos. Node 20+ built-in node:test with --experimental-strip-types executes all 626 tests in under 70 seconds with zero external test runner dependencies.

5. ACCOMPLISHMENTS TILL NOW & PRODUCTION VERIFICATION

Monorepo health, resolved vulnerabilities, and deployment metrics

TEST SUITE RECORD: 100% PASSING

Total Monorepo Tests: 626 tests across 129 test suites
Passing: 626 (100%) | Failing: 0 | Cancelled: 0 | Skipped: 0 | Duration: 68.7 seconds
Packages Verified: @draftpilot/web, @draftpilot/extension, @draftpilot/api

Key Production Crises Diagnosed & Permanently Solved

- > **Fixed Website Login 401 Lockout:** Diagnosed that Kasada BotID was blocking raw browser GET /login document navigations. Updated middleware.ts to allow initial HTML page rendering so human telemetry scripts run. /login and /join now return HTTP 200.
- > **Fixed Extension Timeout Fallback Trigger:** Diagnosed that the extension aborted requests at 12s, which caused reasoning models taking 13-14s to abort and fall back to generic offline templates. Increased extension timeout to 45s and server timeout to 35s.
- > **Fixed "Hi concern," & "Hi Md.,Md.," Greeting Bugs:** Added comprehensive blacklist filters (concern, whomever, whom, party) and eliminated punctuation regex duplication when sender names end in periods (Md.). Added regression test suites.
- > **Implemented SuperAdmin Account Switcher:** Added dedicated "Sign Out / Switch Account" action to AdminSidebar.tsx allowing root administrators to clear credentials and log into customer accounts seamlessly.
- > **Honest Source UI Labeling:** Guaranteed truthful UI badges in the extension sidepanel: [AI Generated] for LLM drafts, [Macro] for grounded macros, and [Fallback Template] (amber) for offline templates.
- > **Automated Extension Build & Distribution:** Configured pnpm build to compile the Vite extension bundle and package draftpilot-extension.zip directly into Next.js public/ for instant one-click customer downloads.

DraftPilot Production Status: **STABLE, OPTIMIZED & DEPLOYED**
Verified on live Vercel production deployment: <https://draftpilot-web.vercel.app>